

Why Zenoh is fast for everyone else and slow for us — measured across the whole landscape

We stood up a byte-faithful replica of the EFDI router (real image, real config, real stack) on isolated hardware, reproduced the slowness, then walked from bare Zenoh to the full load one layer at a time — recording exactly what each thing costs, and whether hardware can buy it away.

Faithful replica

Reproduced

Bare → full attribution

Buyability tested

01 THE ONE-LINE ANSWER

Zenoh's engine is genuinely fast — native round-trip on the real image measured **9 μ s**, right on Zenoh's published spec. Our slowness is **self-inflicted**, and it lives in exactly two things we do that almost nobody else does: we create a huge number of distinct topic keys (cardinality), and we constantly crawl the router with broad tree-walking queries. Neither is expensive alone. Together, on Zenoh's single routing lock, they collide — and no amount of CPU, RAM, or GPU buys you out of it.

02 WHAT WE ACTUALLY ASK ZENOH TO DO

The landscape, from the engine outward. Everything past the router is a *layer of load* we add:

- **The router** — one process, one in-memory table of every declared key ("routing table"), guarded by a single read/write lock.
- **Storages** — ~256 per-tenant RocksDB stores that persist data.
- **Publishers** — tenants + health services (fabric/mesh/time/presence) writing data.
- **Subscribers** — the audit listener, health roll-ups, inspection tools reading data.
- **Crawlers / queriers** — panoscope, the health probe, catch-up queries: tools that walk the *whole* tree (**) to introspect it.
- **Cardinality** — how many *distinct* keys exist. One tenant alone runs ~64k.

03 THE WALK: WHAT EACH LAYER COSTS

Starting from a bare router and adding one layer at a time, holding all else fixed. "Scoped GET" = a cheap single-key admin read (the responsiveness proxy); "walk" = a full tree traversal (what a crawler does). At rest, with no active writes:

LAYER ADDED	SCOPED GET	TREE WALK	ROUTER CPU	VERDICT
bare router (256 storages, empty)	2 ms	17 ms	0%	baseline
+ 6 health publishers	2 ms	16 ms	0.1%	free
+ inspection tools + portal	2 ms	18 ms	0.1%	free
+ 22 tenant publishers	2 ms	19 ms	0.7%	free
+ audit subscriber	2 ms	18 ms	0.8%	free
+ 16 crawlers (no cardinality)	2 ms	22 ms	133%	CPU only, no drag
+ 5,000 keys	2 ms	38 ms	133%	walk grows
+ 15,000 keys	3 ms	227 ms	131%	walk O(N)

The whole service stack is free. Health publishers, tenants, subscribers, inspection tools, the portal — every one of them adds essentially zero drag to the router. This is the surprise that kills the "too many services" theory.

Crawling alone burns CPU but not latency — 16 crawlers push the router to 133% CPU, yet responsiveness stays 2 ms, because with few keys each walk is over in a blink. Cardinality alone makes the tree-walk grow linearly — $O(N)$ in the number of keys (17 → 227 ms as keys go 46 → 15k). That's a real, permanent tax on every broad query, but at rest it doesn't stall anything.

04 THE WEDGE: WHERE IT ACTUALLY BREAKS

Everything above is measured *at rest*. The failure appears the instant you add active writes — new keys being declared — *while* the crawlers are reading. That's the collision on the single routing lock: a writer needs it exclusively, the crawlers are holding it, and the bigger the key table the longer they hold it.

STATE (50K KEYS)	SCOPED GET	TREE WALK
at rest (crawl only, no writes)	2 ms	~300 ms
+ active writes colliding with crawl	650 ms — 7.3 s	6 — 29 s

That is the wedge: responsiveness collapses from 2 ms to seconds, the router stops answering, the watchdog restarts it. It is `keyexpr` cardinality × broad crawling × active write churn, multiplied together on one serial lock — which is why chasing any single culprit (storages, RocksDB, one service) never found it. None of them is the cause; the *interaction* is.

05 CAN WE BUY OUR WAY OUT?

We swept router CPU from 1 to 6 cores at full load. The wedge didn't improve — it got *worse* (650 ms → 7.3 s), because a single lock held serially cannot use extra cores (Amdahl's law), and more box CPU just lets the load push harder. Per dimension:

● RAM

No lever. Whole router RSS is ~160 MB. Not memory-bound.

● GPU

Irrelevant. Routing is lock + pointer work; no GPU path exists.

● CPU — the wedge

No. A serial lock can't parallelize. Wedge persisted/worsened 1→6 cores.

● CPU — steady reads

Constant factor only. Makes each $O(N)$ walk faster; the $O(N)$ trend stays.

● Network / less drop

Real but narrow: fewer mesh reconnects → fewer re-declarations → less write churn (the trigger). Doesn't touch cardinality.

● Architecture

The only real fix. Bound cardinality, shard routing domains, cut broad crawls.

You cannot buy your way out of the wedge. It's structural. What hardware buys is a bigger cushion before the $O(N)$ read cost becomes noticeable — a constant factor, not an escape.

06 THE STORY, IN ONE PARAGRAPH

Zenoh is built to move *messages* — millions per second, microsecond latency — and at that job it is excellent and we barely use 0.01% of its capacity. We are not using it that way. We are using it as a giant, continuously-introspected *directory*: hundreds of thousands of distinct keys, and a fleet of tools that constantly walk the entire directory to build dashboards and topology. Zenoh keeps that directory in one table behind one lock. Reading the whole directory costs more as it grows; writing to it needs everyone else to stop; and doing both at once, at our scale, seizes the lock. Industry doesn't hit this because industry uses the message pipe and keeps the directory tiny. The fix isn't a faster machine — it's asking Zenoh to hold a smaller directory (bound per-tenant cardinality), splitting the directory across independent routers (sharding), and introspecting it less bluntly (scoped queries, not whole-tree crawls on a timer).

Method: byte-faithful EFDI replica on isolated arm64 (real image sha-27c4e9b1 + real 19,921-line config + real service stack). Native RTT 9 μ s. All figures measured, not modeled. CPU-sweep confounded by co-located load generators — directionally conclusive (serial-lock wall), not a clean 1/CPU curve.